终端编程助手：设计总览与项目规格

从零手写的 TypeScript 工程项目

终端编程助手：设计总览与项目规格

- 摘要
- 背景与范围
- 目标与非目标
  - 目标
  - 非目标
- 系统设计
  - 组件与依赖方向
  - 一次任务的数据流
  - 状态所有权与提交边界
  - 工程布局
  - 配置、磁盘与命令行
- 全局规范
  - 术语与可观察性
  - 统一错误合同
  - 默认预算
- 里程碑与交付条件
  - M1：内存中的模型与工具循环
  - M2：会话、恢复、分支与压缩
  - M3：项目上下文与技能
  - M4：文件、命令和 Git 元数据
  - M5：MCP、扩展和 RPC
  - M6：终端界面和端到端集成
- 备选方案与取舍
- 横切关注点
- 验证、演进与待决事项
- 写作参考

终端编程助手：设计总览与项目规格

属性	内容
状态	设计提案，可据此独立实现；本文不代表已有实现
文档版本	1.0，2026-10-22

属性	内容
读者	熟悉 TypeScript、Promise、文件系统和基础测试的开发者
交付物	一个可恢复会话、可执行工具、可扩展、可在终端交互的单用户编程助手
实现语言	TypeScript strict；Node.js 22 或更新的兼容版本；POSIX 平台为验收基线
验证方式	Vitest、临时目录、假模型、内存协议传输和假终端；核心验收无需 API key

摘要

用户在终端提出编程任务，程序把用户消息和历史发送给模型。模型可以请求读文件、修改文件或运行命令；程序校验并执行请求，将结果作为新消息发回模型，直到得到最终答复。会话以追加日志保存，重新启动后可以继续。模型接入、会话存储、工具、资源发现和界面各自独立。

这是一份供人手动编程的项目文档。公共行为由本文和六个里程碑的规格定义；无需阅读其他产品的源码。接口声明用于明确契约，不提供函数实现。你应自行组织控制流、编写算法和测试。

文档分为两层：总览说明问题、范围、架构和取舍；里程碑给出可观察行为、输入输出、失败条件、数据结构、不变量、算法约束和验收示例。每页的“行为规格”是实现必须满足的合同，“详细设计”是推荐的内部组织。可以替换内部设计，但应说明理由并保持行为规格不变。

背景与范围

一次模型调用不足以完成需要观察环境的编程任务。工具循环需要处理多轮调用、取消和错误；文件修改发生后，进程可能在记录结果前退出；长会话会超过上下文预算；命令行界面也不应拥有自己的第二套会话状态。这些问题决定了本项目的模块边界。

本项目面向本地、单用户、单个工作目录。一个进程可以读取许多历史会话，但同一时间只运行一个活动会话。会话的分支是日志上的父子关系，不是 Git 分支。项目约束中的“恢复”指恢复消息与未完成调用的记录，不保证回滚磁盘或远程副作用。

目标与非目标

目标

1. 用户能够输入任务，看到流式文字和工具执行结果，并中断当前运行。
2. 退出后重新打开会话，保留历史、活动分支和上下文配置；不会自动重放可能有副作用的工具。
3. 通过项目说明和按需加载的技能，提供可检查、可复现的上下文。
4. 支持文件工具、命令执行、MCP 工具和本地扩展，使用同一工具执行入口。

5. 同一核心可以连接一次性输出、JSONL RPC 和终端交互三种界面。
6. 每个里程碑具有离线、确定性的验收方法；真实模型不是判断实现正确与否的依据。

非目标

- 多用户服务、远程托管、计费、分布式调度和多代理协作。
- 自动规划模式、向量数据库、跨会话检索或自动学习用户偏好。
- 操作系统级沙箱、恶意扩展隔离、文件系统与会话日志之间的分布式事务。
- 全功能终端模拟器、图片渲染、复杂主题和所有供应商的模型支持。
- 兼容其他项目的存档、字段命名或命令行协议。

系统设计

组件与依赖方向

CLI 是组合根，创建 Session、资源快照、工具和各接入层。Session 依赖 Core；工具和 MCP adapter 实现 Core 定义的端口；Gateway 依赖 Session；TUI 保持领域无关。

Core 不读取会话文件、不遍历资源、不输出终端转义码。Session 是日志唯一写入者。TUI 只接收界面数据，不导入 Core 类型；CLI 将领域事件转成界面操作。Gateway 与 CLI 共用同一个 Session，不得另建循环。下表中的允许依赖即 import 边界。

包	负责的状态	允许依赖	不应负责
`@local/core`	当前运行、消息快照、工具表、两种消息队列	schema 校验库	磁盘、RPC、终端
`@local/session`	日志、活动叶节点、投影、压缩	core、Node 内置	执行文件工具
`@local/resources`	一次扫描生成的不可变资源快照	Node 内置、YAML 解析库	调用模型、写会话
`@local/tools`	工作区路径、文件变更队列	core、Node 内置	修改会话分支
`@local/mcp`	连接、请求映射、工具名称映射	Node 内置；仅 adapter 依赖 core	展示 UI
`@local/gateway`	扩展注册表、RPC 请求编号集合	core、session、mcp、resources	第二份日志
`@local/tui`	编辑器、屏幕行缓存	Node 内置、字符宽度库	直接操作 Agent
`apps/cli`	配置和上述对象的生命周期	所有包	重写包内算法

一次任务的数据流

1. CLI 解析输入并在 idle 时刷新资源快照。
2. Session 先记录资源配置变化，再向 Core 提交用户消息。

3. Core 发出消息事件；Session 在 `message_end` 上先落盘，再允许后续操作继续。
4. Core 将上下文快照交给 `ModelAdapter`。流式片段仅用于显示，最终消息才进入历史。
5. assistant 结束为 `toolUse` 时，Core 校验工具名、参数和钩子，再执行整批工具。
6. tool result 按调用源顺序进入历史并落盘。若未终止，回到模型调用。
7. 纯文本完成、预算耗尽、取消或运行错误结束任务；Session 结清写入，界面回到 idle。

状态所有权与提交边界

消息有唯一 `id`，assistant 内的工具调用有唯一 `callId`。运行有唯一 `runId`，每次模型请求有 `turnId`。不得用数组下标、时间戳或工具名替代标识符。所有消息、事件和日志载荷在发布后视为不可变；实现可采用复制或冻结，不允许订阅者改写历史。

会话记录的提交点是整行追加完成。正常持久化模式在工具开始之前，对包含 tool call 的 assistant 行执行 `datasync`；每条 tool result 也执行 `datasync` 后才继续下一轮。这样进程中断可留下“意图已记录、结果未知”的调用。它仍无法消除磁盘、设备和外部系统的故障，不提供恰好一次执行。

只有 Session 可以改变 Core 的上下文；必须在 idle 时用 `replaceContext`。活动运行拥有开始时捕获的模型、工具和资源快照，`/reload` 不可在运行途中修改它们。

工程布局

使用 npm workspaces。根目录定义 `package.json`、锁文件、`tsconfig.base.json` 和 Vitest 配置。推荐启用 `strict`、`noUncheckedIndexedAccess`、`exactOptionalPropertyTypes`，使用 `NodeNext` 和 ESM。每包有 `src/index.ts`、`test/` 和独立 `tsconfig.json`。禁止从别包的 `src/` 或 `test/` 深路径导入。

里程碑	新增主要文件	单一职责
M1	`core/src/{types,agent,loop,queue,model,tool-runner}.ts`	消息、状态机、队列、模型边界、工具执行
M2	`session/src/{types,store,projection,recovery,compaction,session}.ts`	编解码、日志、分支、修补、压缩、协调
M3	`resources/src/{paths,context,skills,prompt,snapshot}.ts`	路径、说明、技能、序列化、差异检测
M4	`tools/src/{paths,mutation-queue,truncate,read,write,edit,grep,find,ls,bash,git}.ts`	七种工具和 Git 元数据
M5	`mcp/src/{jsonrpc,client,adapter,names,transport,stdio,in-memory}.ts`	协议和适配

里程碑	新增主要文件	单一职责
M5	`gateway/src/{extensions,rpc-types,rpc,framer}.ts`	扩展宿主和外部控制
M6	`tui/src/{terminal,renderer,editor,keys,text}.ts`	终端抽象、差分、编辑器和宽度
各阶段	`apps/cli/src/{main,wire,print,interactive}.ts`	按已完成阶段逐步添加入口

测试辅助实现放在各包 `test/helpers/`；CLI 演示用假模型放在 `apps/cli/src/demo-model.ts`，不可从发布包引用测试文件。目录表是组织建议，不要求机械拆成每个文件，也不要求提前创建空包。

配置、磁盘与命令行

默认数据根目录为 `~/.terminal-agent`；`TERMINAL_AGENT_HOME` 可以覆盖。工作区根是启动时 `realpath(cwd)`。会话目录使用该绝对路径的 UTF-8 SHA-256 完整十六进制值，避免路径替换编码发生碰撞。

```
~/.terminal-agent/
  settings.json
  skills/
  extensions/
  mcp.json
  sessions/<workspaceHash>/<sessionId>.jsonl
<workspace>/.terminal-agent/
  skills/
  extensions/
  mcp.json
<workspace>/AGENTS.md
```

项目资源默认未信任。只有显式 `--trust-project` 才可读取项目技能、项目扩展和项目 MCP 配置。存在 `.git` 或某个目录不能隐式授予信任。用户级扩展和命令同样具有本机权限，配置它们相当于选择执行本地代码。文件工具的路径检查不是对 bash 或扩展的沙箱。

用法	行为
`terminal-agent --print "任务"`	一次运行；最终 assistant 文本写 stdout；诊断写 stderr
`terminal-agent --print`	读 stdin 至 EOF 作为一个任务；空白输入报 `INVALID_INPUT`
`terminal-agent --resume <sessionId>`	恢复当前工作区指定会话；不自动重放工具
`terminal-agent --rpc`	stdin/stdout 仅传 JSONL，见 M5
`terminal-agent`	M6 后进入交互；非 TTY 提示使用 `--print` 或 `--rpc`，退出 2
`--demo`	使用确定性演示模型；不访问网络

`--print` 和 `--rpc` 互斥。未知参数、无效输入和缺失配置退出 2；运行失败退出 1；用户中断退出 130；成功退出 0。默认模式只在 M6 接通。M1 使用内存上下文；M2 后除测试显式 `persist:false` 外均落盘。

真实接入需要实现一个 `ModelAdapter`，在 M6 端到端阶段提供一种供应商适配即可。密钥只从进程环境读取，不写消息、日志或 RPC 状态。无法提供适配时应明确处于 demo 模式，不能把假模型当真实助手。

全局规范

术语与可观察性

术语	精确定义
run	从一次 `prompt` 或 `continue` 到一个 `agent_end`
turn	run 内一次模型请求及其对应工具批；不是一条用户输入
transcript	Core 当前使用的消息序列，不含流式半成品
entry	会话日志中一条带父指针的记录；不是所有 entry 都是消息
projection	从活动叶节点回溯并应用编辑、配置和压缩后生成的模型上下文
committed	一条完整日志行的写入成功；关键执行边界已完成同步

时间一律用毫秒整数，文本 UTF-8，ID 默认 `crypto.randomUUID()`，测试可注入 clock 和 ID 工厂。排序明确使用 Unicode 码点顺序，不能依赖文件系统枚举或 locale。

统一错误合同

业务错误使用 `{code, message, details?}`，`details` 只含可 JSON 序列化数据；测试断言 `code` 与结构字段，不匹配操作系统报错原文。表中是整个项目的错误命名空间。

code	触发范围	必须保证
`INVALID_INPUT` / `INVALID_CONFIG`	参数、配置、schema	未产生任务或副作用
`BUSY`	idle 限定 API 在运行中调用	原运行不变
`MODEL_ERROR` / `MODEL_PROTOCOL`	模型网络失败或非法流	不执行不完整工具调用
`UNKNOWN_TOOL` / `INVALID_ARGS` / `TOOL_BLOCKED`	工具执行前	execute 次数为 0
`TOOL_FAILED` / `TIMEOUT`	执行中	保存失败结果，不假定回滚
`ABORTED`	取消	不再启动新的模型请求或工具
`RUN_LIMIT`	turn 上限	有界终止，保留已完成结果
`PERSISTENCE` / `CORRUPT_SESSION`	日志 I/O、结构损坏	停止继续执行副作用
`PATH_ESCAPE` / `NOT_FOUND` / `CONFLICT`	文件工具	失败时不覆盖目标

code	触发范围	必须保证
`DISCONNECTED` / `PROTOCOL_ERROR`	MCP	结清 pending 请求
`INTERNAL`	未分类的程序缺陷	运行结束，诊断带 runId，不暴露密钥

公开 API 参数校验错误、`BUSY` 和 `PERSISTENCE` 拒绝 Promise。已开始的运行中的模型/工具/取消错误收敛为事件及 `RunResult`。Session 写失败具有更高优先级，不能伪装成普通 tool error。RPC 用结构化 response；CLI 用 stderr；TUI 用独立错误区。

默认预算

配置	默认值	合法范围
`maxTurns`	32	正整数
`toolTimeoutMs`	120000	正整数
`contextWindow`	模型声明；demo 为 8192	正整数
`reserveTokens`	1024	正整数，须小于 contextWindow
`keepRecentTokens`	4096	正整数；保留组可能超过此软目标
`maxToolOutputBytes`	51200	正整数
`maxToolOutputLines`	2000	正整数
`rpcLineBytes`	1048576	每条入站记录上限

所有设置在启动时校验一次。压缩后若仍不能容纳一次请求，返回 `INVALID_INPUT` 并指出上下文过大，不无限压缩或静默删除用户消息。token 估计仅用于保守预算，真实供应商可能有不同计数。

里程碑与交付条件

以下次序按依赖确定。每个里程碑的链接指向完整规格，规格包含操作级语义和设计解释。总览不重复字段定义，发生冲突时先修订文档再编码，不靠猜测选择一种行为。

M1：内存中的模型与工具循环

用户结果：在 demo 模式输入问题，模型调用假工具后输出答案，可以中断。

规格与详细设计：Core。必须覆盖 prompt、continue、两种队列、订阅、取消、模型流、工具批、循环终止。数据结构包括消息判别联合、运行状态、工具注册表、事件信封和 FIFO。设计模式为状态机、策略、观察者及命令队列，各模式的用途和局限见该页。

交付：core、demo 模型和 `--print`；不创建 session 文件。退出门槛：M1-A01 至 M1-A12 全部离线通过，含并发完成顺序与取消屏障。

M2：会话、恢复、分支与压缩

用户结果：重启恢复历史，切换历史分支，压缩上下文而不改写原始日志。

规格与详细设计：Session。覆盖日志编码、损坏处理、append、checkout、context edit、悬空调用修补、摘要事务。数据结构包括 entry 树、ID 索引、活动叶节点和投影快照。设计模式为追加事件日志、投影、写前记录与工作单元；此处不把日志冒称完整的事务型事件溯源系统。

交付：session 包、`--resume`、CLI 统一经 Session 调用 Core。退出门槛：M2-A01 至 M2-A12 通过；注入日志写失败后没有工具执行。

M3：项目上下文与技能

用户结果：项目说明自动加入上下文；技能只以索引暴露，可显式调用并刷新。

规格与详细设计：Resources。覆盖发现边界、优先级、frontmatter、诊断、快照和 `/skill` 命令。数据结构包括来源链、技能映射、诊断和规范化 prompt。设计模式为资源库、不可变快照、渐进加载和组合根。

交付：resources 包、`--trust-project`、`/reload` 和 `/skill:<name>`。退出门槛：M3-A01 至 M3-A10 通过；未信任时不遍历项目资源目录。

M4：文件、命令和 Git 元数据

用户结果：模型可读取、搜索、精确编辑工作区，运行命令并看到截断结果。

规格与详细设计：Tools。七个工具各有参数、输出、失败与运行时间规格；另定义路径校验、写队列、UTF-8 截断和 worktree 发现。设计模式为 Command、策略、按工作区串行执行器和适配器。

交付：tools 包，替换 demo 假工具；demo 与测试仍可注入假工具。退出门槛：M4-A01 至 M4-A14 通过，失败编辑保留原字节，命令超时后完成进程清理。

M5：MCP、扩展和 RPC

用户结果：接入本地 MCP 服务；扩展注册工具/钩子；其他进程通过 JSONL 驱动同一会话。

规格与详细设计：MCP 与 Gateway。覆盖握手、请求关联、断线取消、分页、名称映射、扩展注册事务、RPC 请求与事件顺序。数据结构包括 pending Map、双向名称表、扩展注册批、命令判别联合。设计模式为适配器、端口与适配器、责任链和请求关联。

交付：mcp、gateway 和 `--rpc`。退出门槛：M5-M01 至 M5-M10、M5-G01 至 M5-G12 通过；stdout 无非协议文本，abort 不被长 prompt 阻塞。

M6：终端界面和端到端集成

用户结果：在终端编辑输入、查看流式输出与工具状态；退出时终端恢复。

规格与详细设计：TUI。覆盖按键解析、编辑器、视口、差分刷新、宽度、终端控制字符和异常清理。数据结构包括 grapheme 缓冲、编辑光标、屏幕缓存和界面 view model。设计模式为表示模型、单向数据流和依赖倒置。

交付：tui、默认交互模式、一种真实模型适配器及其配置说明。退出门槛：M6-A01 至 M6-A12 与所有先前验收通过。离线完成一条“读文件 → 修改 → 再读 → 最终回复”的端到端轨迹；真实供应商只做可选 smoke test，不用于确定性断言。

备选方案与取舍

决策	选定方案及原因	备选及放弃原因
模型依赖	小型 `ModelAdapter` 端口，便于模拟流和取消	核心绑定 SDK 会把供应商状态泄漏到存档
会话存储	单写者 JSONL，易检查、易制造中断夹具	SQLite 更利于查询和事务，但遮蔽本项目要练习的投影与恢复
分支	entry 父指针 + 显式 checkout 记录	复制完整日志容易，但分支共用历史和压缩更难追踪
界面	核心事件转换为 TUI view model	TUI 直接改 Core 状态会产生第二条写入路径
工具调度	只读批并行；出现变更类工具则整批串行	所有工具并行增加磁盘竞争；所有工具串行损失读性能
学习组织	里程碑内同时给规格与设计理由	只给类名不足以独立实现；完整代码会取消设计练习

横切关注点

安全与隐私：会话含用户输入、文件内容和命令输出，默认以目录 0700、文件 0600 创建（POSIX）。不记录环境变量全集、密钥和完整异常请求头。模型返回的终端文本必须转义控制字符；项目说明不能改变程序级权限。

可靠性：单会话文件只有一个进程写入，使用排他锁文件，见 M2。工具副作用不可回滚；中断恢复产生“结果未知”记录，不能声称操作没发生。取消是协作式，扩展未响应取消时也不能谎报已经停止。

性能：流式 update 合并到约 30fps 显示，最终结果不能丢。日志载入与分支投影线性于记录量；工具名查找与协议请求关联平均 O(1)。大文件搜索是流式，不先把整个工作区读进内存。

可观测性：结构化诊断带 runId、turnId、callId 和错误 code。状态快照可导出，但隐藏密钥。运行统计包括请求数、工具数、耗时、取消和压缩次数；不要求遥测服务。

验证、演进与待决事项

每包提供 `test`、`typecheck` 脚本；根 `npm test` 跑全部离线测试，`npm run typecheck` 检查全部包，`npm run demo -- --print "任务"` 演示最小闭环。这些是待实现项目的验收命令，不代表当前文档仓库已有对应代码。使用 deferred Promise 控制并发次序，fake clock 控制超时，不以 sleep 或模型输出文案作正确性依据。

日志和 RPC 各有版本号。1.0 不实现旧版迁移，未知版本明确拒绝。未来格式变化需新增迁移设计、夹具和回退策略；不得默默以当前格式读取。关键决策和规范更新先写到对应文档，再更新实现和验收编号。

当前实现所需的协议和行为决策均已明确。未来独立评审的议题是 Windows 命令进程树清理、远程 MCP、图片消息和多进程读写；它们不阻塞本项目基线，也不应在验收时临时加入。

写作参考

本文采用 Design Docs at Google 的背景、目标、方案、取舍及横切关注点组织方式，并用 CS61B Gitlet 的操作规格思路明确输入、效果、失败条件和复杂度。这两份材料是写作参考；本项目的行为和存储格式由本文独立定义。